

Task 17: Inter-Service Communication using REST

AIM

To enable inter-service communication using REST-based communication by creating two microservices where one consumes the REST API exposed by the other, handling responses and failures gracefully.

ALGORITHM

Step 1: Create Microservice-A (Provider) that exposes a REST API endpoint (e.g., GET /api/products).

Step 2: Create Microservice-B (Consumer) that calls Microservice-A's API using an HTTP client.

Step 3: Implement error handling in the consumer for scenarios such as timeout, 404, and 500 errors.

Step 4: Use retry logic or circuit breaker pattern for resilience.

Step 5: Parse and display the response received from the provider service.

Step 6: Test both services by running them on different ports and verifying communication.

PROGRAM

```
// ----- Microservice-A (Provider) - providerService.js -----  
  
const express = require('express');  
const app = express();  
  
const products = [  
  { id: 1, name: 'Laptop', price: 75000 },  
  { id: 2, name: 'Mouse', price: 500 },  
  { id: 3, name: 'Keyboard', price: 1500 }  
];  
  
app.get('/api/products', (req, res) => {  
  res.json({ success: true, data: products });  
});  
  
app.get('/api/products/:id', (req, res) => {
```

```

const product = products.find(p => p.id === parseInt(req.params.id));
if (!product) return res.status(404).json({ success: false, message: 'Product not found'
});
res.json({ success: true, data: product });
});

app.listen(3001, () => console.log('Provider Service running on port 3001'));

// ----- Microservice-B (Consumer) - consumerService.js -----
const express = require('express');
const axios = require('axios');
const app = express();

const PROVIDER_URL = 'http://localhost:3001';

app.get('/fetch-products', async (req, res) => {
  try {
    const response = await axios.get(`${PROVIDER_URL}/api/products`, { timeout: 5000 });
    res.json({ source: 'Microservice-A', products: response.data.data });
  } catch (error) {
    if (error.code === 'ECONNREFUSED') {
      res.status(503).json({ error: 'Provider service is unavailable' });
    } else if (error.response) {
      res.status(error.response.status).json({ error: error.response.data.message });
    } else {
      res.status(500).json({ error: 'Unexpected error: ' + error.message });
    }
  }
});

app.get('/fetch-product/:id', async (req, res) => {
  try {
    const response = await axios.get(`${PROVIDER_URL}/api/products/${req.params.id}`, {
      timeout: 5000 });
    res.json({ source: 'Microservice-A', product: response.data.data });
  } catch (error) {
    if (error.response && error.response.status === 404) {
      res.status(404).json({ error: 'Product not found in provider service' });
    } else {

```

```
res.status(500).json({ error: 'Failed to fetch from provider: ' + error.message });
}
}
});

app.listen(3002, () => console.log('Consumer Service running on port 3002'));
```

OUTPUT

Console Output (Provider - port 3001):

Provider Service running on port 3001

Console Output (Consumer - port 3002):

Consumer Service running on port 3002

GET http://localhost:3002/fetch-products

```
{  "source":    "Microservice-A",    "products":    [{ "id":1,"name":"Laptop","price":75000},
{"id":2,"name":"Mouse","price":500}, {"id":3,"name":"Keyboard","price":1500}] }
```

GET http://localhost:3002/fetch-product/1

```
{ "source": "Microservice-A", "product": { "id":1,"name":"Laptop","price":75000} }
```

GET http://localhost:3002/fetch-product/99 (Not Found):

```
{ "error": "Product not found in provider service" }
```

When Provider is down:

```
{ "error": "Provider service is unavailable" }
```

RESULT

Successfully implemented REST-based inter-service communication between two microservices. The consumer service gracefully handles success responses, 404 errors, connection failures, and timeouts from the provider service.